I. PerfMLOps: Zero-Human in the Loop Performance Testing Framework - Framework Design

A. System Architecture

This framework includes triggering load tests on JMeter as soon as the Python API is deployed after the change. Some performance issues were intentionally induced in the JMeter test script to evaluate the framework's ability to identify bottlenecks. Engineers receive an email after the successful deployment and execution of the JMeter load test. The events generated from JMeter were automatically fed to Good Analytics and then to Google's BigQuery for further analysis. Engineers receive a daily report on the events captured in Google Analytics via email. During the initial setup, the BigQuery ML Model "Boosted tree Classifier" was trained, and the model was later used to identify the number of bottlenecks and their root causes. Google's cloud function service was created to send an email summarizing the tests with bottlenecks and root causes identified by the ML model. These steps are represented in one figure, as shown in Fig. 1.

B. Implementation of Python API

The application layer is built as a RESTful API using Python's Flask framework. This API has a dual-purpose design: a functioning CRUD service for structured data and a performance simulation environment that replicates realistic system characteristics. This design's modularity makes it ideal for loading structured event data into Google Analytics and then exporting it to BigQuery for predictive modeling using SQL ML.

This API has two main functionalities:

1) Basic CRUD operations on an in-memory dataset.
2) Performance testing endpoints are designed to simulate various server behaviors.

- Health Check Endpoint
  - "/" → Returns a simple JSON response indicating that the API is active. This is primarily used to monitor the API's availability.
- CRUD Endpoints
  The API simulates a lightweight in-memory database (items =) and provides the standard Create, Read, Update, Delete (CRUD) functionality:
  - POST /items/ → Creates a new item, ensuring unique names.
  - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
  - GET /items/"name" → Retrieves an item by name, returning 404 if not found.
  - PUT /items/"name" → Updates an existing item with new data.
  - DELETE /items/"name" → Deletes an item by name.

  This design offers a simple yet extensible data interaction model suitable for experimentation and integration with downstream services.
- Performance Testing Endpoints
  To evaluate system resilience and simulate different conditions, the API provides specialized endpoints:
  - /fast → Returns an immediate response, representing minimal latency.
  - /slow → Introduces a 2-second delay before responding, simulating a slow endpoint.
  - /random → Introduces a random delay (0.1–3.0s) to emulate unpredictable response times.
  - /unstable → Has a 20% probability of failure (returns HTTP 500) and otherwise succeeds, modeling unstable network or server conditions.
- Deployment This API was deployed on the local machine using the command

```
python3 PTAPI.py
```

C. JMeter Script

This JMeter script was designed to perform CRUD operations: Add, Get, Update, and Delete items using the "HTTP Request" and "HTTP Header Manager" elements, which support 50 TPS for 1 hour. Test data for the script was injected using CSV Data Set Config element. Induced response times and response codes to Google Analytics using JSR223 requests for pre/post-processing. A GA request was created to send event data; Google Analytics results are collected as listeners for analysis.

D. Google Analytics 4 Setup

Integrated JMeter to send events to GA4 by creating a data stream for the measurement ID and API secret which are passed as part of the API request to feed data into GA from JMeter via API. JMeter sends events to GA using the end-point (https://www.google-analytics.com/mp/collect?measurement_id=G-XXXXXXX&api_secret=your_secret) and parameters to send in the body of the request along with "event name". Google Analytics then reads the data based on custom events created under the data stream created based on "event name" [**?**].

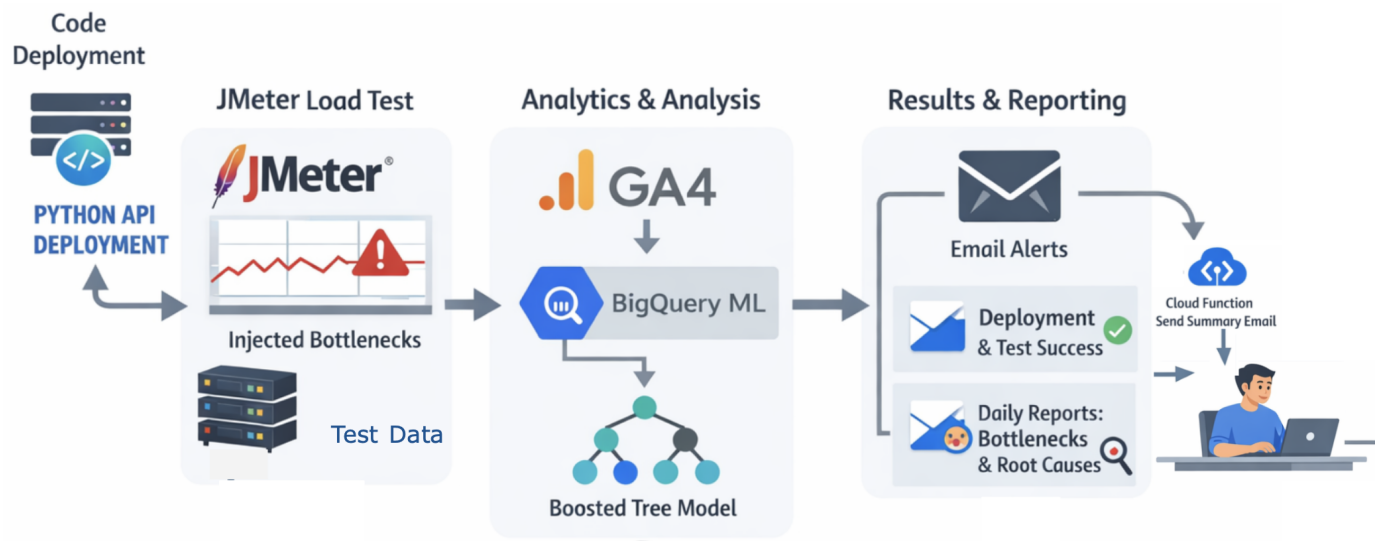

Fig. 1. System Architecture Used in this Study

E. BigQuery Setup

After an account has been created in BigQuery, GA can be integrated with the BigQuery project using the BigQuery link in the Admin-product link section. Once linked, Google Analytics automatically creates a dataset in BigQuery and inserts it into the tables.

F. BigQuery ML Setup

ML models can be created and trained with simple SQL queries. The process typically starts with preparing the dataset in the BigQuery table and creating the model with the desired model type and method. The regression type of model method is typically used to identify bottlenecks. Based on input features, the target column, and training options, the model is created. Once created, the model is stored inside BigQuery and evaluated in BigQuery. The model can later be used to identify bottlenecks and the root causes of those bottlenecks.

G. Cloud Run Function setup

A service was created using Google Cloud Functions, where a Python script was developed to connect to BigQuery, retrieve the query results, and send the summarized output via email. Required dependencies were specified in the requirements.txt file and installed during deployment. Upon successful deployment, the function generated an HTTP endpoint URL, which triggers the execution of the Python code when invoked.